DESARROLLO CLOUD NATIVE I

Informe de evidencias prácticas RabbitMQ

Mensajería asíncrona, Spring Boot, React/Vite y Docker

Integrantes	Bastián Balladares / Bastián Perez / Camila Concha
Asignatura	Desarrollo Cloud Native I
Experiencia	Experiencia 2 – RabbitMQ
Fecha	05 de octubre de 2026

Índice

1. Introducción y objetivos
2. Arquitectura y conceptos aplicados
3. Preparación del entorno
4. RabbitMQ con Docker Compose
5. Primera actividad: cola, productor y consumidor
6. Segunda actividad: exchange, bindings y routing keys
7. Productor REST y consumidores especializados
8. Pruebas INFO, WARNING y ERROR
9. Integración con frontend React/Vite
10. Manejo de errores y recuperación
11. Persistencia temporal de mensajes
12. Routing key no configurada
13. Evidencias finales en RabbitMQ Management
14. Resultados y conclusiones

El documento reúne evidencias de ambas carpetas entregadas: el conjunto de capturas “Evidencias” y la carpeta “rabbitmq”, incluyendo material de la primera y segunda actividad. Cada captura se acompaña de una explicación breve para contextualizar qué se comprobó y por qué es relevante.

1. Introducción y objetivos

La actividad tuvo como propósito implementar y verificar un sistema de mensajería asíncrona con RabbitMQ. Se trabajó progresivamente desde la validación del entorno hasta una integración completa entre React/Vite, Spring Boot y RabbitMQ, comprobando tanto el flujo normal de mensajes como escenarios de falla y recuperación.

Objetivos desarrollados

- Verificar Docker, Java y Maven como base del laboratorio.
- Levantar RabbitMQ mediante Docker Compose y acceder a RabbitMQ Management.
- Implementar una cola básica con productor y consumidor en Spring Boot.
- Configurar un DirectExchange con múltiples colas, bindings y routing keys.
- Exponer un endpoint REST para publicar mensajes desde clientes externos.
- Integrar un frontend React/Vite para enviar logs INFO, WARNING y ERROR.
- Comprobar almacenamiento temporal de mensajes cuando el consumidor está detenido.
- Validar el comportamiento de una routing key sin binding asociado.

La estrategia de trabajo fue incremental: cada componente se verificó antes de avanzar al siguiente. Esto permitió aislar errores y comprobar el comportamiento observable del broker en cada etapa.

2. Arquitectura y conceptos aplicados

La solución final utiliza dos protocolos: HTTP entre el frontend y Spring Boot, y AMQP entre Spring Boot y RabbitMQ. RabbitMQ actúa como broker intermediario y desacopla la publicación del procesamiento de los mensajes.

React / Vite – localhost:5173
↓ HTTP POST /log
Spring Boot – localhost:8080
↓ RabbitTemplate / AMQP
RabbitMQ – logs_direct_exchange
↓ Routing keys: INFO · WARNING · ERROR
all_logs_queue / errors_only_queue → consumidores

Conceptos clave

- Producer: componente que genera y publica mensajes.
- Exchange: recibe mensajes y decide a qué cola enviarlos.
- Routing key: clave utilizada por el exchange para decidir la ruta.
- Binding: relación entre un exchange y una cola asociada a una binding key.
- Queue: almacena mensajes hasta que un consumidor pueda procesarlos.
- Consumer: componente que recibe y procesa mensajes desde una cola.

3. Preparación del entorno

Antes de desarrollar la aplicación se comprobaron las herramientas requeridas. Las capturas siguientes provienen de la carpeta rabbitmq y documentan la validación del entorno realizada al inicio del laboratorio.



Figura 1. Comprobación de Docker instalado y disponible.

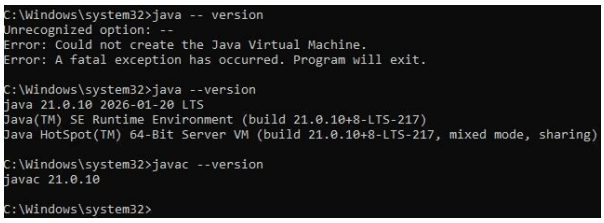


Figura 2. Comprobación de Java para ejecutar Spring Boot.



Figura 3. Comprobación de Maven, utilizado para compilar y ejecutar el proyecto Spring Boot.

Estas verificaciones reducen la posibilidad de atribuir a RabbitMQ problemas originados por dependencias o herramientas faltantes. Una vez confirmadas, se procedió a levantar el broker.

4. RabbitMQ con Docker Compose

RabbitMQ se ejecutó en un contenedor Docker utilizando una definición Compose. Se publicaron los puertos 5672 para AMQP y 15672 para la interfaz web de administración.

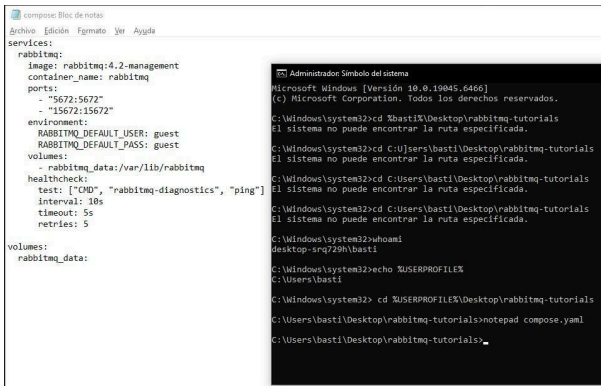


Figura 4. Creación de la configuración Docker Compose.



Figura 5. Verificación del contenedor mediante docker compose ps.

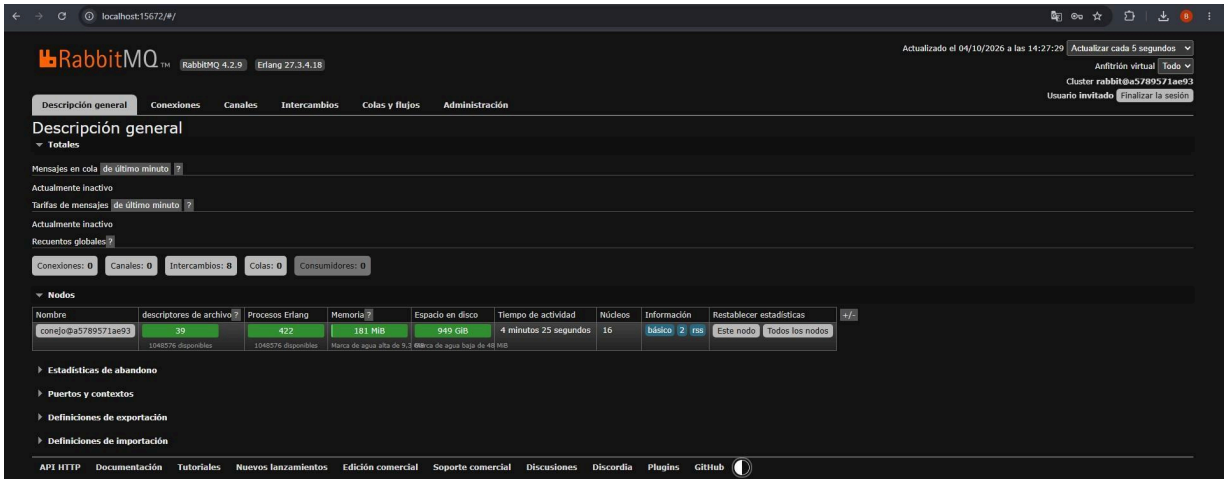


Figura 6. Acceso a RabbitMQ Management, confirmando que el broker se encuentra operativo.

La interfaz de administración fue utilizada posteriormente para inspeccionar colas, exchanges, bindings, consumidores y mensajes pendientes.

5. Primera actividad: cola, productor y consumidor

La primera actividad permitió comprobar el patrón básico de mensajería. Se creó el proyecto Spring Boot, se configuró la conexión con RabbitMQ y se implementaron los componentes Sender y Receiver.

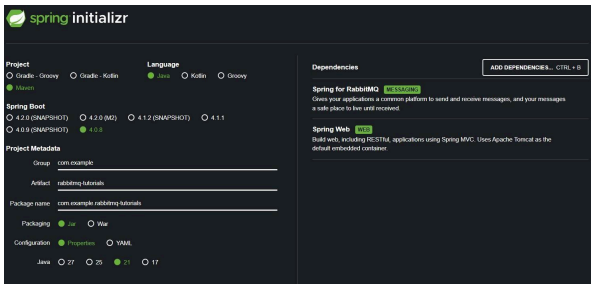


Figura 7. Generación del proyecto Spring Boot.

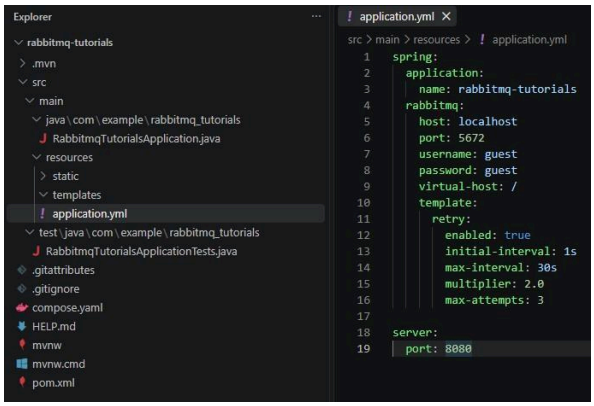


Figura 8. Configuración de conexión en application.yml.

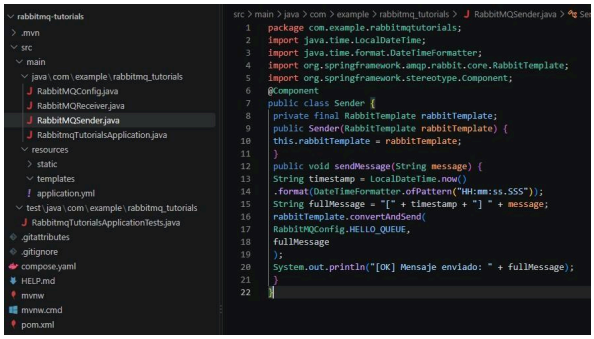


Figura 9. Implementación del productor Sender.

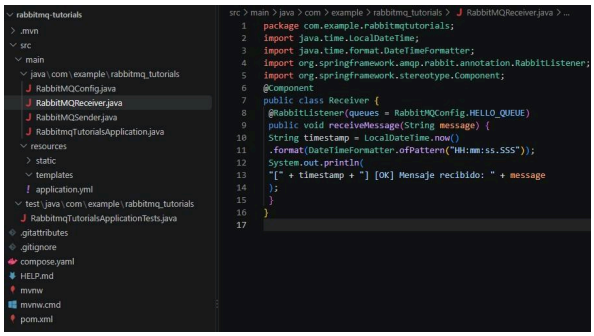


Figura 10. Implementación del consumidor Receiver.

```
1. Enviar mensaje
2. Salir
Seleccione: 1
Mensaje: Hola RabbitMQ
[OK] Mensaje enviado: [15:30:12.512] Hola RabbitMQ

1. Enviar mensaje
2. Salir
Seleccione: [15:30:12.523] [OK] Mensaje recibido: [15:30:12.512] Hola RabbitMQ
```

Figura 11. Ejecución de la primera prueba de envío y recepción de mensaje.

Con esta prueba se validó el flujo productor → RabbitMQ → cola → consumidor. Esta base se reutilizó en la segunda actividad, donde el enrutamiento se volvió más específico.

6. Segunda actividad: exchange, bindings y routing keys

En la segunda actividad se amplió la topología con logs_direct_exchange, all_logs_queue y errors_only_queue. El exchange es de tipo direct, por lo que compara la routing key del mensaje con las binding keys configuradas.

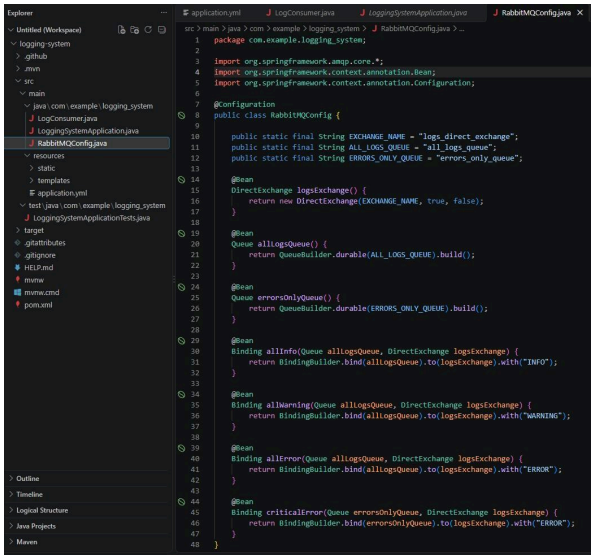


Figura 12. Configuración del DirectExchange, colas y bindings.

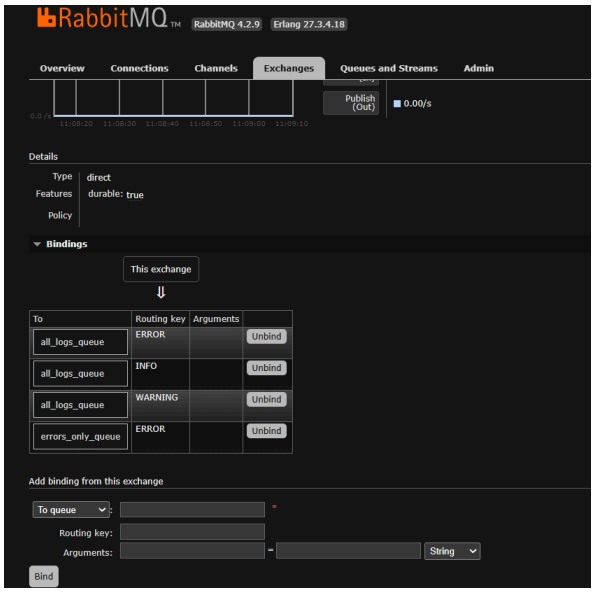
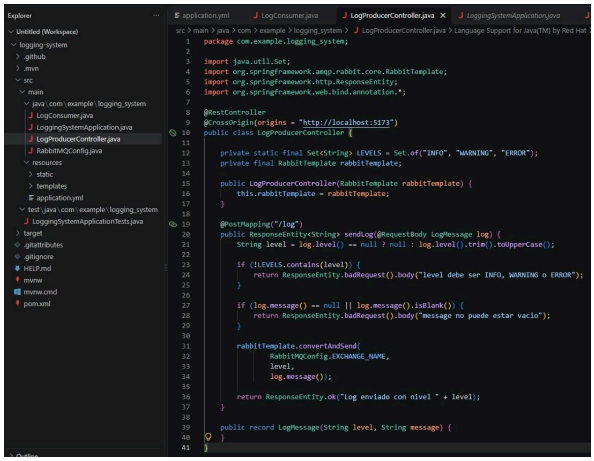


Figura 13. Verificación visual de los bindings en RabbitMQ Management.

La cola all_logs_queue posee bindings para INFO, WARNING y ERROR. La cola errors_only_queue posee un binding adicional para ERROR. Por ello un mensaje ERROR se entrega a ambas colas, mientras INFO y WARNING solo llegan a la cola general.

7. Productor REST y consumidores especializados

Spring Boot expone un endpoint REST que recibe el nivel y el contenido del log. RabbitTemplate publica el mensaje en logs_direct_exchange utilizando el nivel como routing key. Los consumidores procesan las colas de acuerdo con su función.



```java
package com.example.logging_system;

import java.util.Set;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@CrossOrigin(origins = "http://localhost:5173")
public class LogProducerController {

    private static final Set<String> LEVELS = Set.of("INFO", "WARNING", "ERROR");
    private final RabbitTemplate rabbitTemplate;

    public LogProducerController(RabbitTemplate rabbitTemplate) {
        this.rabbitTemplate = rabbitTemplate;
    }

    @PostMapping("/log")
    public ResponseEntity<String> sendLog(@RequestBody LogMessage log) {
        String level = log.level() == null ? null : log.level().trim().toUpperCase();

        if (!LEVELS.contains(level)) {
            return ResponseEntity.badRequest().body("level debe ser INFO, WARNING o ERROR");
        }

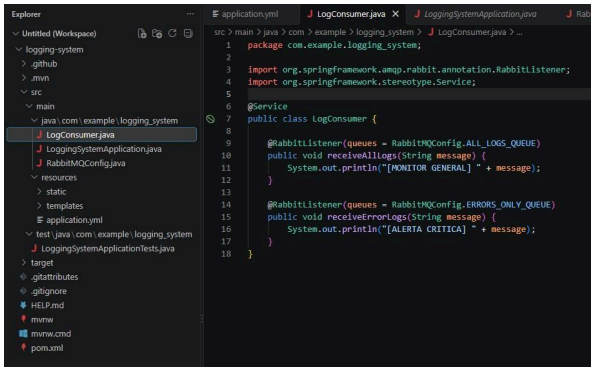
        if (log.message() == null || log.message().isBlank()) {
            return ResponseEntity.badRequest().body("message no puede estar vacio");
        }

        rabbitTemplate.convertAndSend(
                RabbitMQConfig.EXCHANGE_NAME,
                level,
                log.message());

        return ResponseEntity.ok("Log enviado con nivel " + level);
    }

    public record LogMessage(String level, String message) {
    }
}
```

Figura 14. Controlador productor que publica mensajes mediante RabbitTemplate.



```java
package com.example.logging_system;

import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Service;

@Service
public class LogConsumer {

    @RabbitListener(queues = RabbitMQConfig.ALL_LOGS_QUEUE)
    public void receiveAllLogs(String message) {
        System.out.println("[MONITOR GENERAL] " + message);
    }

    @RabbitListener(queues = RabbitMQConfig.ERRORS_ONLY_QUEUE)
    public void receiveErrorLogs(String message) {
        System.out.println("[ALERTA CRITICA] " + message);
    }
}
```

Figura 15. Consumidor de logs configurado con @RabbitListener.

Este diseño evita que el productor tenga que conocer directamente a cada consumidor. El productor pública una vez y la topología del broker determina qué componentes deben recibir cada mensaje.

8. Pruebas INFO, WARNING y ERROR

Se ejecutaron pruebas diferenciadas para confirmar la matriz de enrutamiento. Las evidencias de esta sección combinan capturas de la carpeta rabbitmq y del conjunto Evidencias.



Figura 16. Envío de un mensaje INFO.



Figura 17. Envío de un mensaje WARNING.



Figura 18. Envío de un mensaje ERROR.

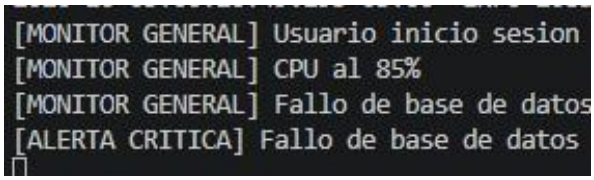


Figura 19. ERROR procesado por el monitor general y la alerta crítica.

Routing key	Monitor general	Alerta crítica
INFO	Sí	No
WARNING	Sí	No
ERROR	Sí	Sí

Los resultados confirman que el enrutamiento depende de los bindings del exchange y no de una decisión manual del productor.

9. Integración con frontend React/Vite

Posteriormente se creó un frontend React/Vite para reemplazar las pruebas manuales por una interfaz de usuario. El formulario permite seleccionar INFO, WARNING o ERROR, escribir un mensaje y enviarlo mediante HTTP POST al backend.

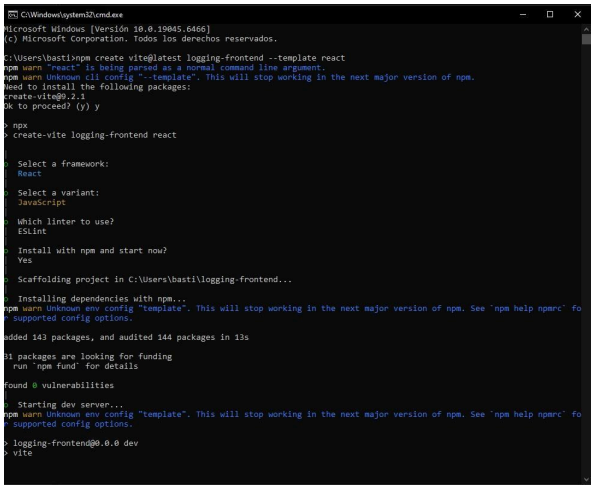


Figura 20. Creación del frontend React/Vite.



Figura 21. Integración del frontend con el endpoint del backend.

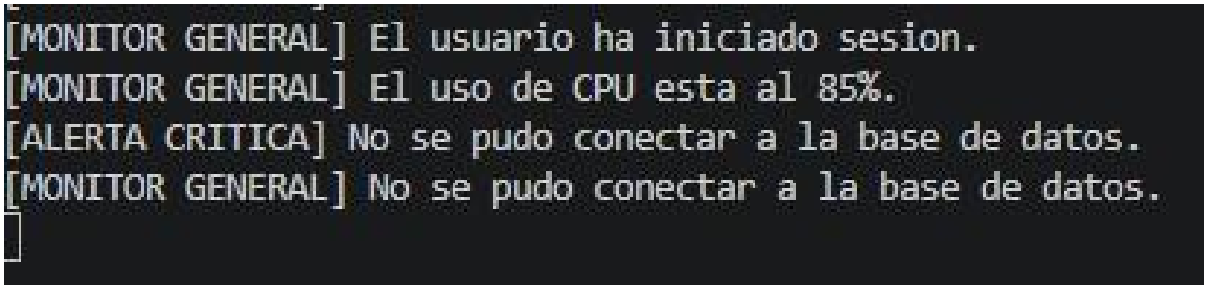


Figura 22. Respuestas obtenidas durante las pruebas de integración.

Para permitir la comunicación entre localhost:5173 y localhost:8080 se configuró CORS en el backend. La aplicación quedó integrada de extremo a extremo: React → Spring Boot → RabbitMQ → consumidores.

10. Manejo de errores y recuperación

El frontend incorpora manejo de errores mediante try/catch. Para comprobarlo se detuvo Spring Boot de forma intencional y se intentó enviar un mensaje desde React.



Figura 23. Con el backend detenido, el frontend informa que el registro no pudo enviarse y conserva el mensaje para permitir un reintento.

Después de volver a levantar Spring Boot, el mismo mensaje fue reenviado correctamente. Este comportamiento mejora la experiencia de uso y permite distinguir una falla de comunicación de un error de enrutamiento dentro de RabbitMQ.

11. Persistencia temporal de mensajes

Para demostrar el desacoplamiento temporal, Spring Boot fue detenido y se publicó un mensaje INFO directamente desde RabbitMQ Management. Como no existía un consumidor activo, el mensaje permaneció almacenado en la cola.

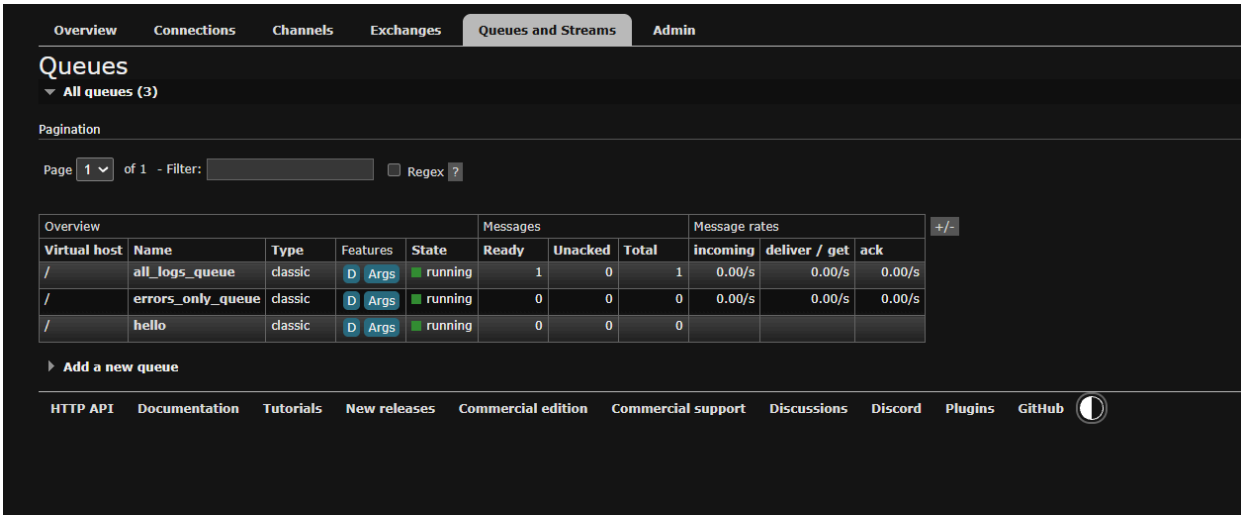


Figura 24. all_logs_queue muestra Ready = 1, Unacked = 0 y Total = 1 mientras Spring Boot está detenido.

Al reiniciar Spring Boot, el consumidor se reconectó y procesó automáticamente el mensaje pendiente. La cola volvió a Ready = 0. Esta prueba demuestra una ventaja esencial de la mensajería asíncrona: productor y consumidor no necesitan estar disponibles al mismo tiempo.

12. Routing key no configurada

Como prueba final de enrutamiento se utilizó la routing key DEBUG, que no posee ningún binding en logs_direct_exchange. RabbitMQ recibió la publicación, pero no encontró una cola de destino.

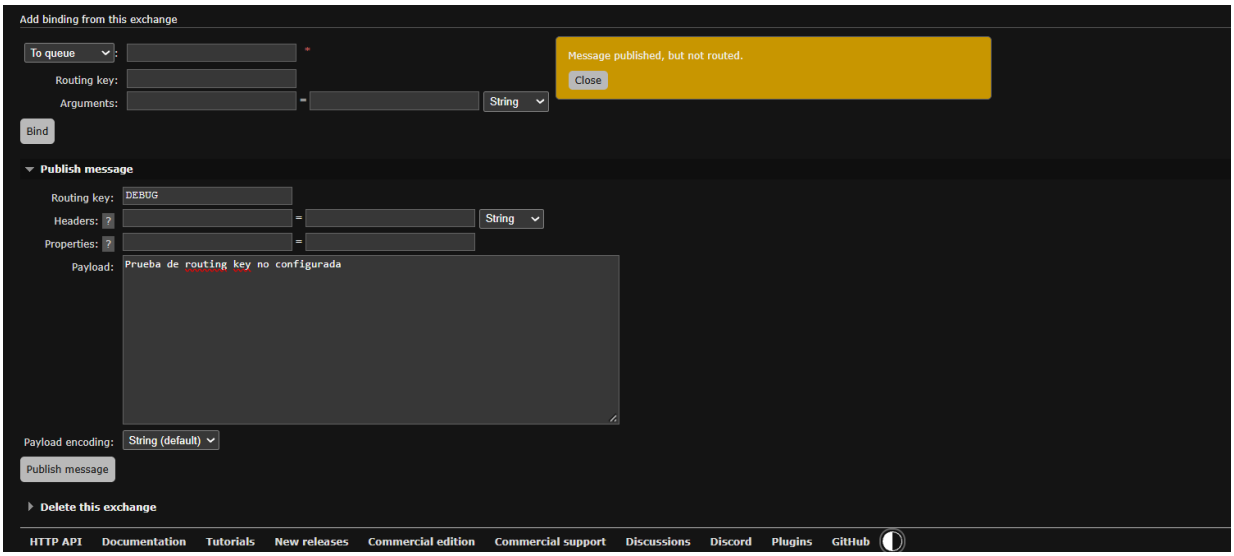


Figura 25. RabbitMQ informa "Message published, but not routed" al publicar con DEBUG.

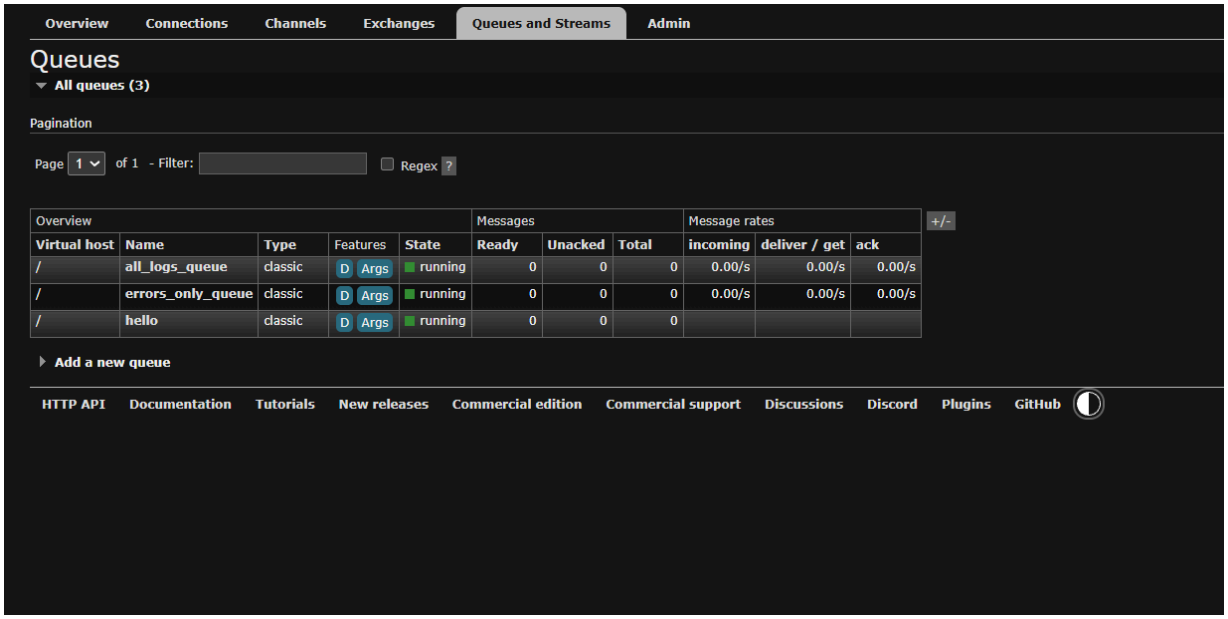


Figura 26. Las colas permanecen en cero, confirmando que DEBUG no fue enrutado.

La prueba confirma el comportamiento esperado de un DirectExchange: la coincidencia entre routing key y binding key debe ser exacta.

13. Evidencias finales en RabbitMQ Management

La interfaz de RabbitMQ permitió comprobar el estado general de la solución una vez integrados backend, frontend y consumidores.

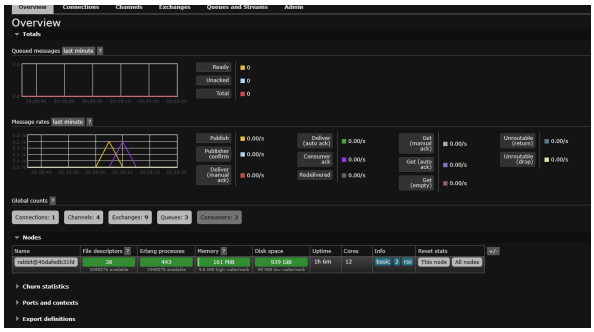


Figura 27. Vista general con conexiones, exchanges, colas y consumidores activos.

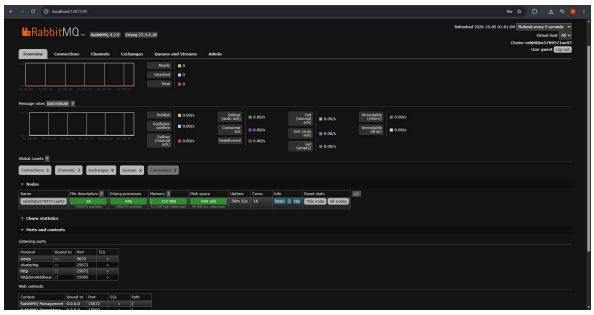


Figura 28. Resultados observados desde la interfaz de RabbitMQ.

Estas evidencias complementan las salidas de Spring Boot: mientras la terminal demuestra el procesamiento de los consumidores, RabbitMQ Management permite inspeccionar la topología y el estado del broker.

14. Resultados y conclusiones

Resultados principales

- Entorno de trabajo validado con Docker, Java y Maven.
- RabbitMQ desplegado mediante Docker Compose y administrado desde su interfaz web.
- Flujo básico productor-cola-consumidor implementado y comprobado.
- DirectExchange configurado con dos colas y cuatro bindings.
- Enrutamiento de INFO, WARNING y ERROR validado experimentalmente.
- Endpoint REST integrado con RabbitTemplate.
- Frontend React/Vite conectado al backend mediante CORS.
- Manejo de errores comprobado con el backend fuera de servicio.
- Mensaje almacenado en Ready mientras no existía consumidor y procesado tras la recuperación.
- Routing key DEBUG comprobada como no enrutable al no existir binding asociado.

Conclusión

El laboratorio permitió aplicar de manera progresiva los componentes fundamentales de RabbitMQ y observar su comportamiento real. La primera actividad validó la comunicación básica; la segunda incorporó un modelo de enrutamiento más cercano a un escenario de aplicación, con separación entre registros generales y alertas críticas.

La integración final demuestra un flujo completo desde una interfaz React hasta consumidores Spring Boot, utilizando RabbitMQ como intermediario. Las pruebas de falla, recuperación y mensajes no enrutados aportan evidencia adicional de que no solo se implementó el caso exitoso, sino que también se comprendieron situaciones relevantes para la operación y diagnóstico de una arquitectura basada en mensajería asíncrona.

Las capturas incluidas fueron seleccionadas de ambas carpetas entregadas, combinando evidencias del proceso de trabajo de los integrantes y las comprobaciones finales realizadas sobre la solución.

- Configuración que habilita el reconocedor automático y el reintento con backoff exponencial (3 intentos).



Foto 1: Se actualiza el yaml

Explicación:

Muestra la clase de configuración en el código fuente (Spring Boot / Java). Se observa la declaración de los Beans para la cola principal (Queue), el intercambio (DirectExchange o TopicExchange), los parámetros del Dead Letter Exchange (DLX) y los Bindings que conectan la cola con el Exchange.

- Se agrega la cola principal (orders.queue), la cola muerta (orders.dlq), los intercambiadores (exchanges) y las reglas de enrutamiento (bindings).

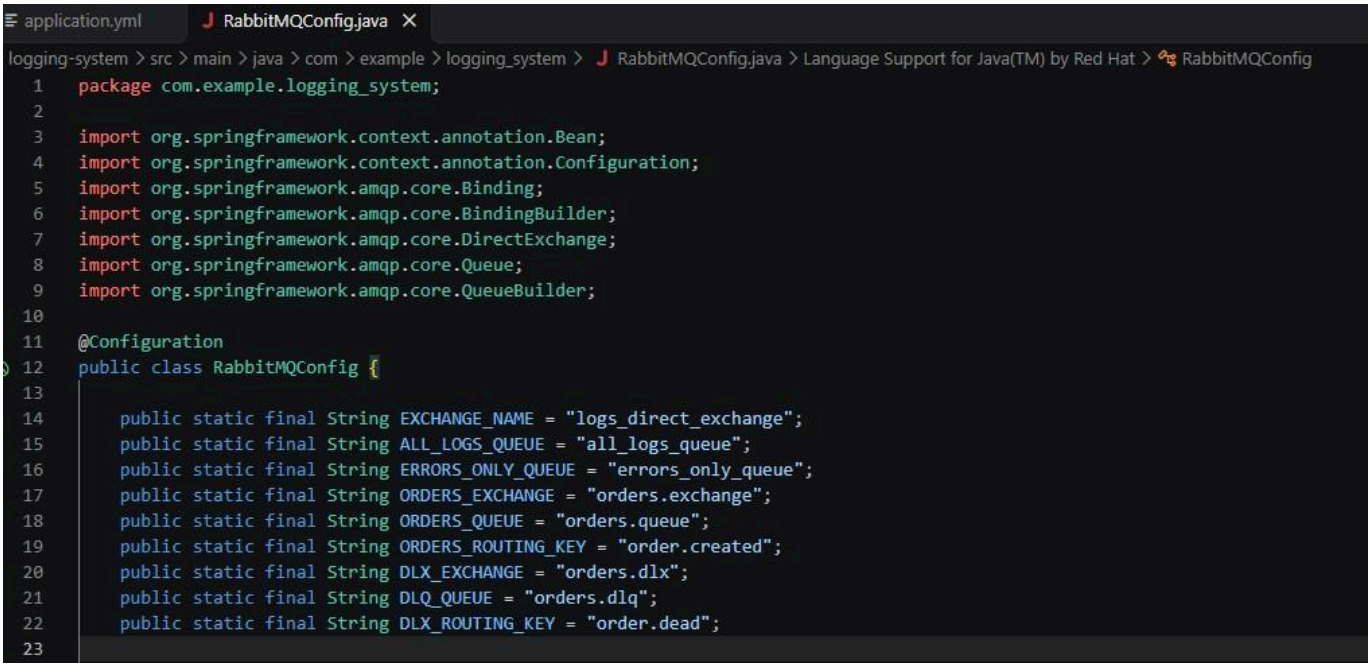


Foto 2: Se actualiza rabbitmqconfig

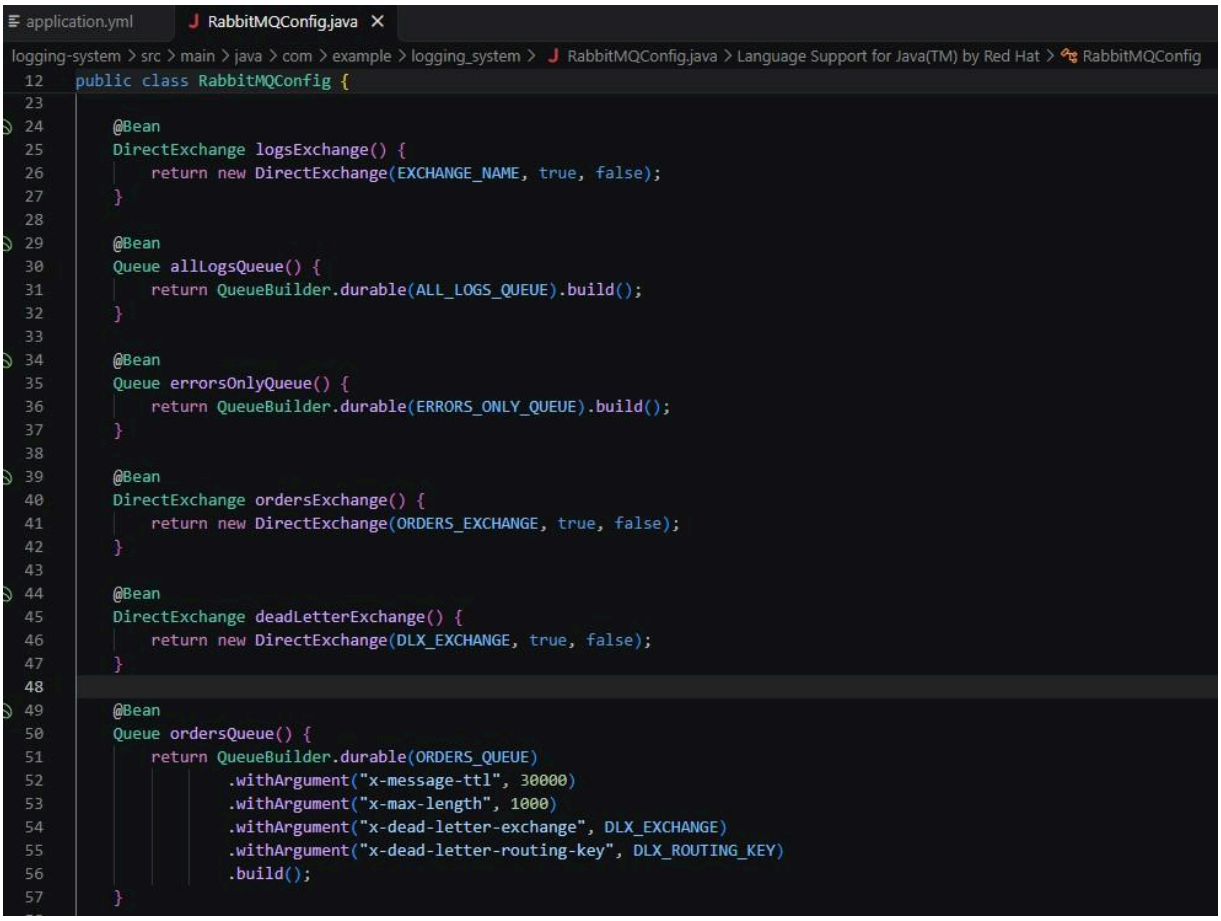


Foto 2.1: Segunda captura de rabbitmqconfig

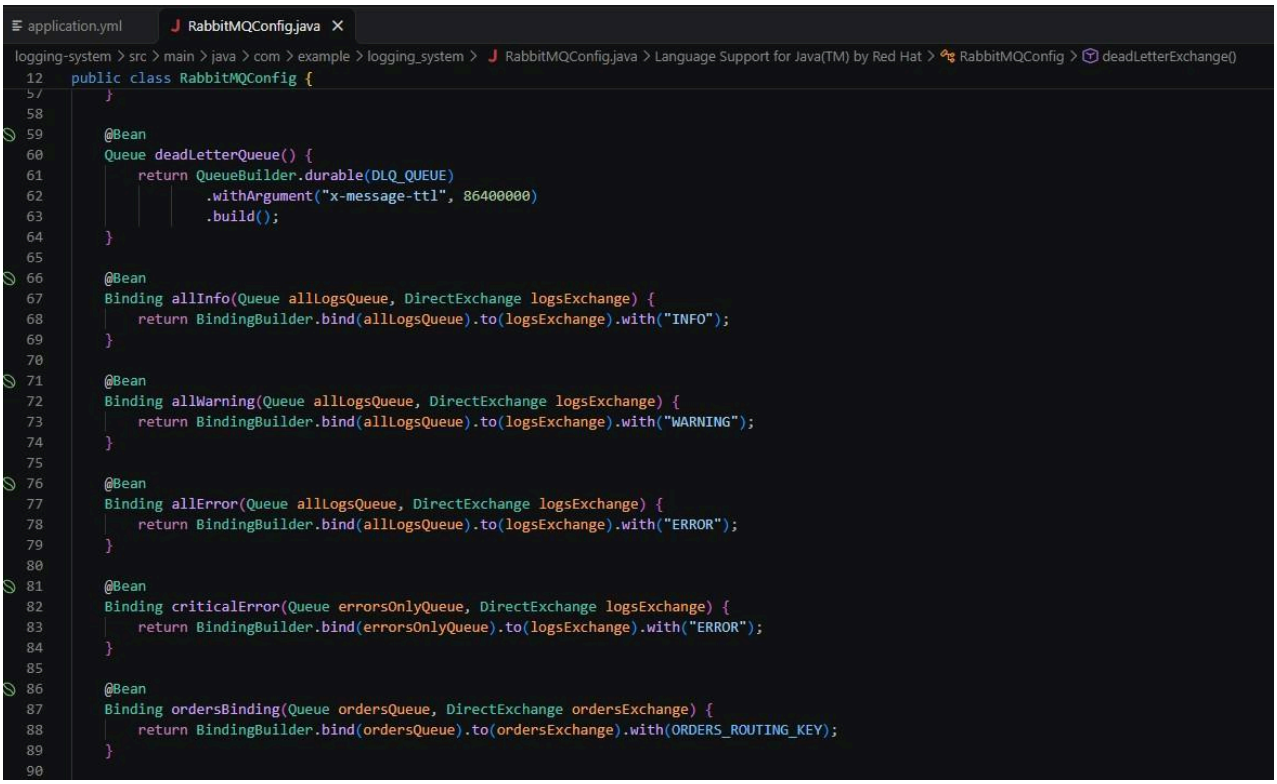


Foto 2.2: Tercera captura de rabbitmqconfig

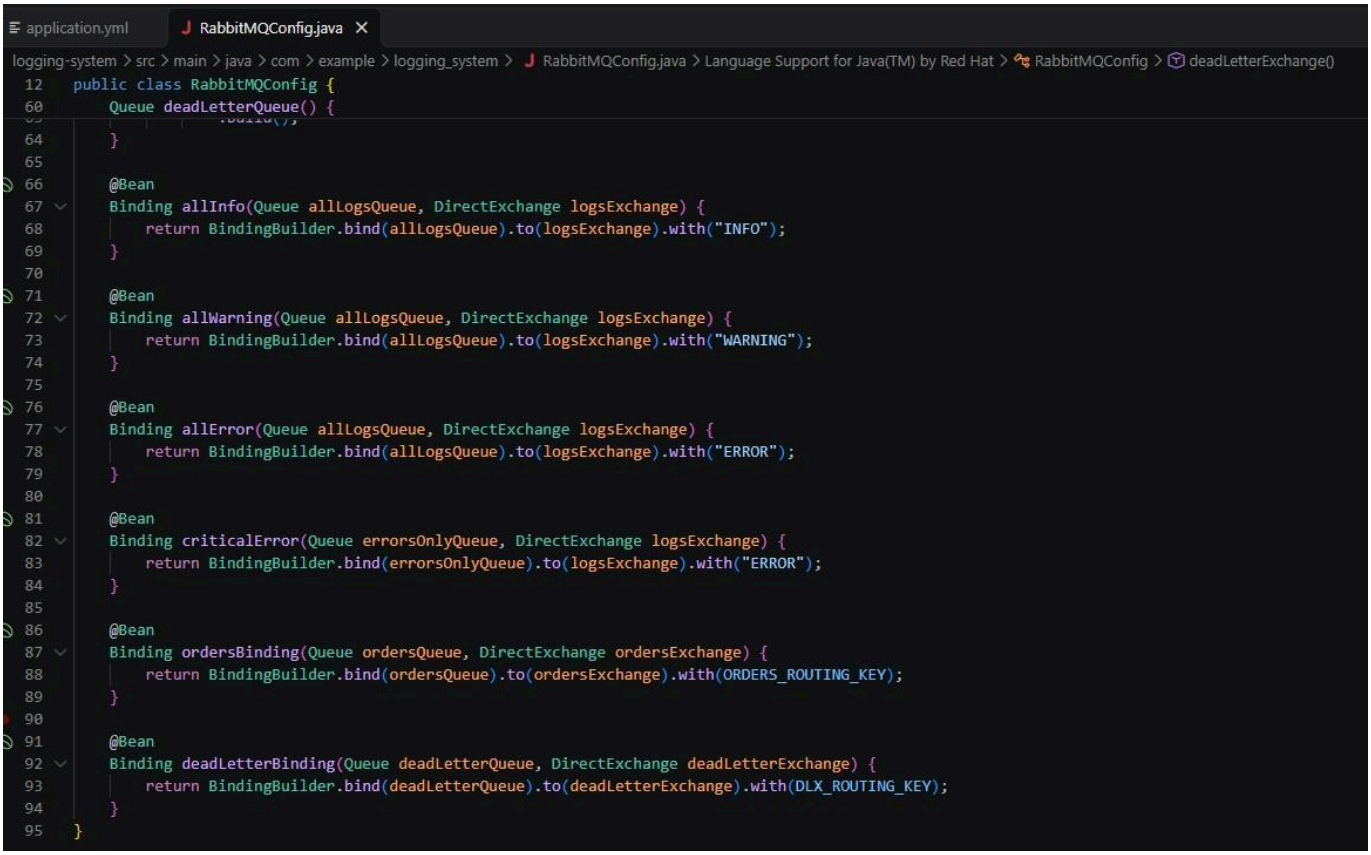


Foto 2.3: Cuarta captura de rabbitmqconfig

Implementación del servicio Productor de mensajes (Publisher)

Explicación:

Muestra el código o servicio encargado de publicar eventos en RabbitMQ utilizando la plantilla de mensajería (RabbitTemplate). Se evidencia la serialización del objeto de la orden a formato JSON y su posterior envío al Exchange correspondiente.

- Service que procesa las órdenes de la cola principal y simula el fallo determinista al detectar la palabra "FAIL".

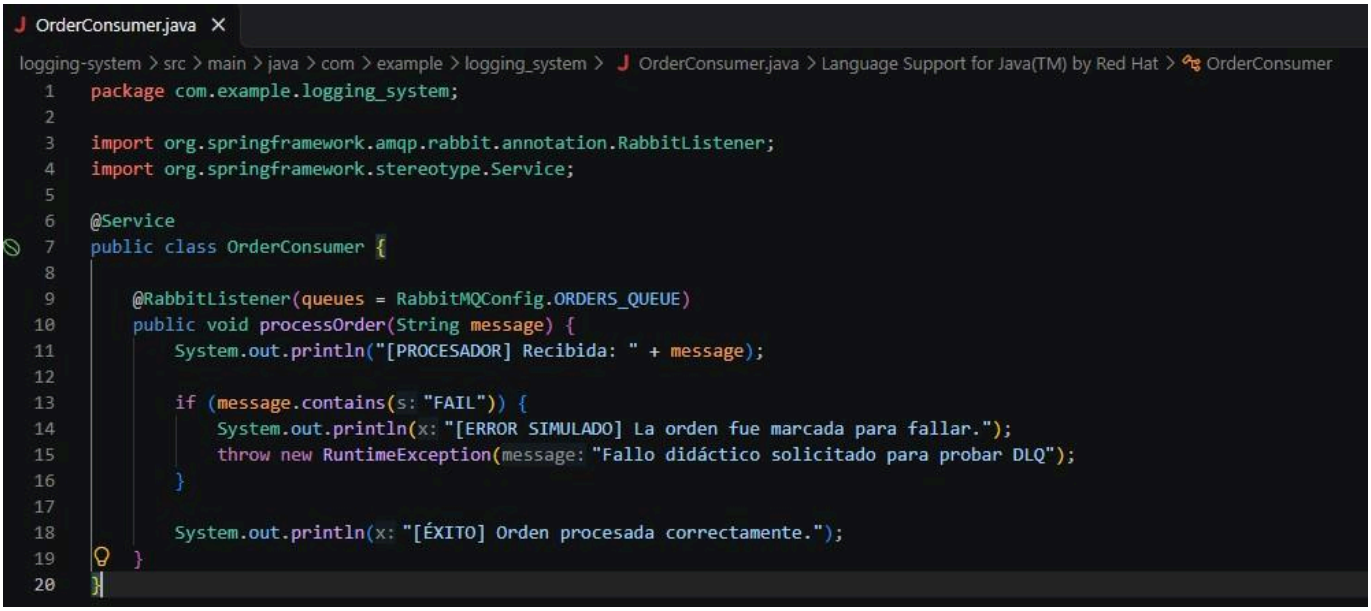


Foto 3: OrderConsumer

Implementación del servicio Consumidor (Listener)

Explicación:

Muestra el método anotado con @RabbitListener encargado de escuchar la cola principal. Se observa la recepción del mensaje, la lógica de negocio para procesar la orden y el bloque de control para capturar excepciones en caso de datos inválidos.

- Controlador REST que ofrece los endpoints para enviar mensajes (/send), verificar conectividad (/status) y consultar las métricas de la DLQ en tiempo real (/metrics).

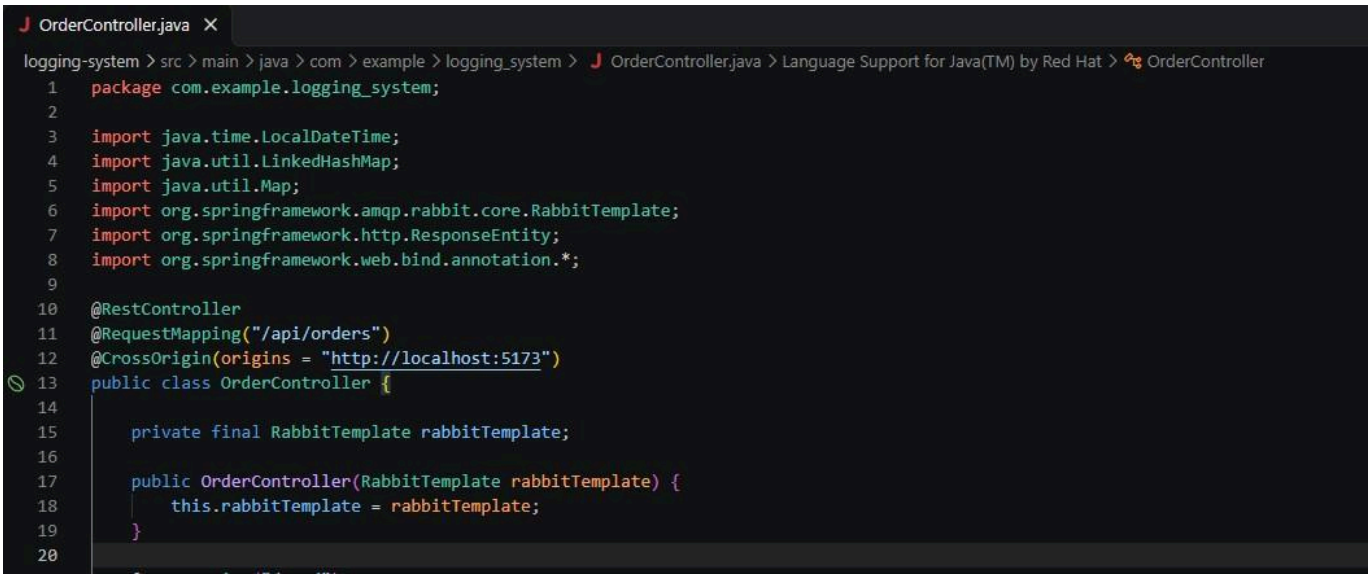


Foto 4: orderController

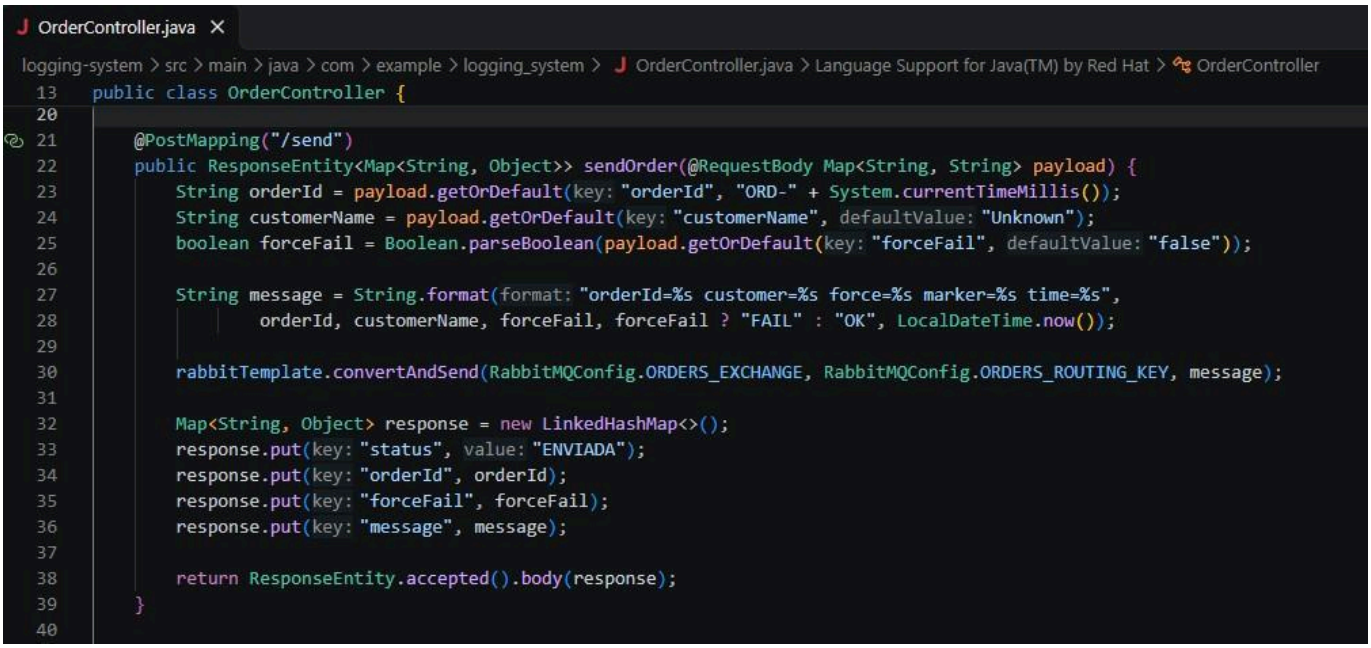


Foto 4.1: Segunda captura de ordercontroller

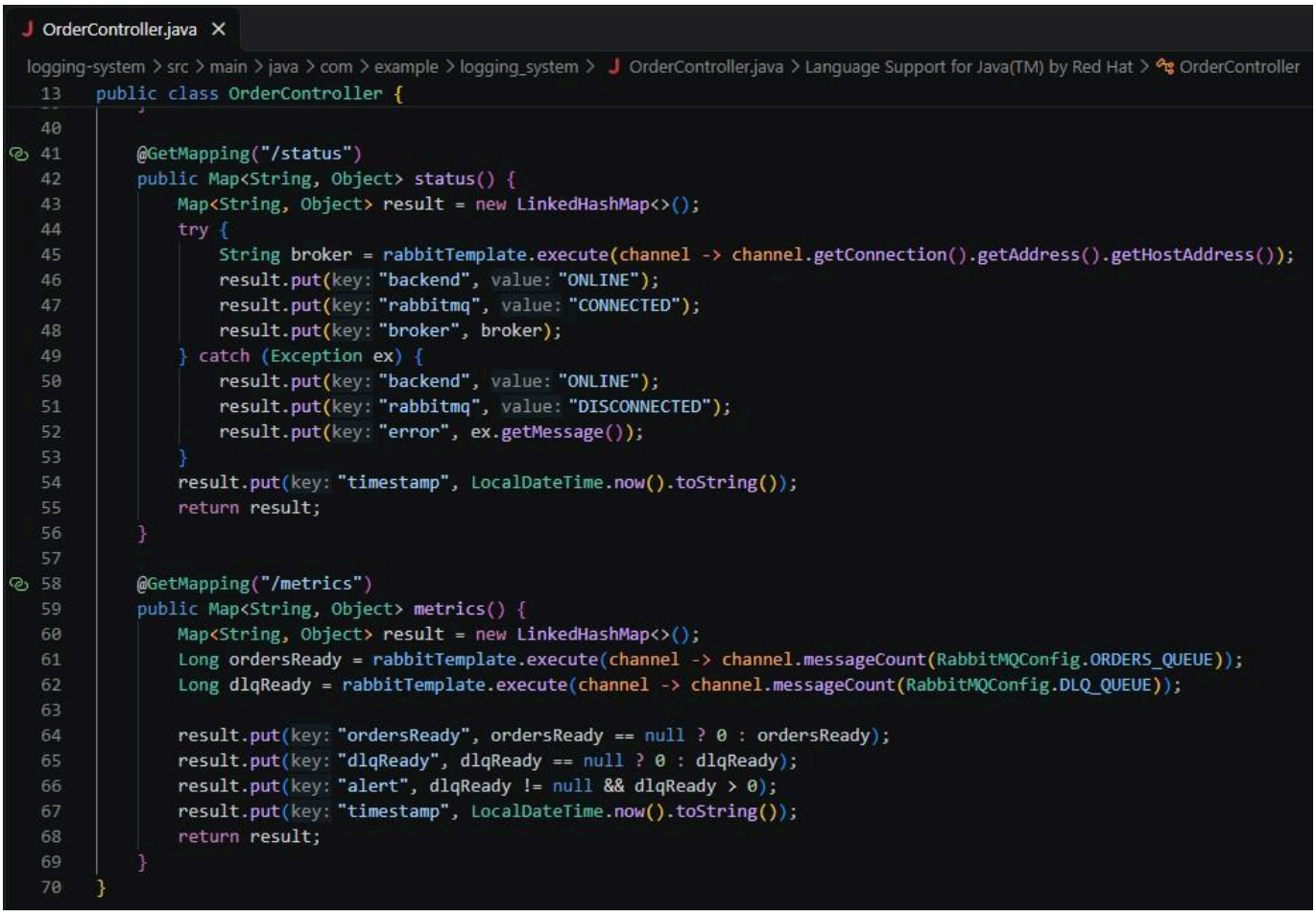


Foto 4.2: Tercera captura de ordercontroller

Actualización del código del Frontend

Explicación:

Muestra las modificaciones realizadas en el código fuente del Frontend (componentes UI, formularios o servicios API). Se evidencia la integración de la lógica para enviar solicitudes de órdenes al Backend y la gestión del feedback visual para el usuario durante el envío del evento a RabbitMQ.

App.jsx

logging-frontend > src > App.jsx > ...

```jsx
import { useEffect, useState } from 'react';
import './App.css';

const API_ORDERS = 'http://localhost:8080/api/orders';
const API_LOGS = 'http://localhost:8080/api/logs'; // Endpoint de la actividad anterior

function App() {
  // Estados para la Actividad de DLX / DLQ
  const [backend, setBackend] = useState('CHECKING');
  const [metrics, setMetrics] = useState({ ordersReady: 0, dlqReady: 0, alert: false });
  const [lastOrderResponse, setLastOrderResponse] = useState(null);

  // Estados para la Actividad Anterior de Logs
  const [logMessage, setLogMessage] = useState('');
  const [logType, setLogType] = useState('INFO');
  const [lastLogResponse, setLastLogResponse] = useState(null);

  // --- MÉTODOS ACTIVIDAD DLX / DLQ ---
  async function refreshMetrics() {
    try {
      const [statusRes, metricsRes] = await Promise.all([
        fetch(`${API_ORDERS}/status`),
        fetch(`${API_ORDERS}/metrics`)
      ]);
      const statusData = await statusRes.json();
      const metricsData = await metricsRes.json();

      setBackend(statusData.rabbitmq === 'CONNECTED' ? 'CONNECTED' : 'DISCONNECTED');
      setMetrics(metricsData);
    } catch (error) {
      setBackend('DISCONNECTED');
    }
  }

```

Foto 5: Actualización del frontend

App.jsx

logging-frontend > src > App.jsx > ...

```
7   function App() {
34
35    useEffect(() => {
36      refreshMetrics();
37      const id = setInterval(refreshMetrics, 2000);
38      return () => clearInterval(id);
39    }, []);
40
41    async function sendOrder(forceFail) {
42      const body = {
43        orderId: `ORD-${Date.now()}`,
44        customerName: forceFail ? 'Cliente DLQ' : 'Cliente OK',
45        forceFail: String(forceFail)
46      };
47
48      const response = await fetch(`${API_ORDERS}/send`, {
49        method: 'POST',
50        headers: { 'Content-Type': 'application/json' },
51        body: JSON.stringify(body)
52      });
53
54      const data = await response.json();
55      setLastOrderResponse(data);
56
57      setTimeout(refreshMetrics, 800);
58      setTimeout(refreshMetrics, 3500);
59    }
60
```

Foto 5.1: Segunda captura del frontend

App.jsx

logging-frontend > src > App.jsx > App > handleSendLog

```
7   function App() {
60
61    // --- metodos de la actividad de logs ---
62    async function handleSendLog(e) {
63      e.preventDefault();
64      try {
65        const response = await fetch(API_LOGS, {
66          method: 'POST',
67          headers: { 'Content-Type': 'application/json' },
68          body: JSON.stringify({
69            message: logMessage,
70            type: logType,
71            timestamp: new Date().toISOString()
72          })
73        });
74        const data = await response.json();
75        setLastLogResponse(data);
76        setLogMessage('');
77      } catch (err) {
78        setLastLogResponse({ error: 'No se pudo conectar con el endpoint de logs' });
79      }
80    }
```

Foto 5.2: Tercera captura del frontend

App.jsx

logging-frontend > src > App.jsx > App

```
 7  function App() {
62    async function handleSendLog(e) {
79    }
80  }
81
82  return (
83    <main className="container">
84      <h1>Panel Integrado - Logging & RabbitMQ DLQ</h1>
85
86      {/* ================ modulo de la nueva actividad dlx y dlq ================ */}
87      <section className="card">
88        <h2>Pruebas DLX / DLQ (Nueva Actividad)</h2>
89        <p>Estado Backend/RabbitMQ: <b>{backend}</b></p>
90        <p>Mensajes listos en cola principal: <b>{metrics.ordersReady}</b></p>
91        <p>Mensajes en DLQ: <b>{metrics.dlqReady}</b></p>
92
93        {metrics.alert && (
94          <div className="alert">
95            ALERTA: existen mensajes en orders.dlq. Revise RabbitMQ Management y los logs del backend.
96          </div>
97        )}
98
99        <div className="actions" style={{ marginTop: '16px' }}>
100         <button onClick={() => sendOrder(false)}>Enviar orden exitosa</button>
101         <button onClick={() => sendOrder(true)}>Enviar orden que debe ir a DLQ</button>
102         <button onClick={refreshMetrics}>Actualizar métricas</button>
103       </div>
104
105       {lastOrderResponse && (
106         <div style={{ marginTop: '16px' }}>
107           <h4>Última respuesta HTTP (Órdenes):</h4>
108           <pre>{JSON.stringify(lastOrderResponse, null, 2)}</pre>
109         </div>
110       )}
111     </section>
112
```

Foto 5.3: Cuarta captura del frontend

App.jsx

logging-frontend > src > App.jsx > App

```jsx
function App() {
            </div>
          )}
        </section>

        {/* ================ modulo de la actividad anterior de logs ================ */}
        <section className="card">
          <h2>Módulo de Logs (Funcionalidad Anterior)</h2>
          <form onSubmit={handleSendLog} style={{ display: 'flex', flexDirection: 'column', gap: '12px' }}>
            <div>
              <label style={{ display: 'block', marginBottom: '4px' }}>Nivel de Log:</label>
              <select value={logType} onChange={(e) => setLogType(e.target.value)} style={{ padding: '8px', width: '100%' }}>
                <option value="INFO">INFO</option>
                <option value="WARN">WARN</option>
                <option value="ERROR">ERROR</option>
              </select>
            </div>
            <div>
              <label style={{ display: 'block', marginBottom: '4px' }}>Mensaje:</label>
              <input
                type="text"
                value={logMessage}
                onChange={(e) => setLogMessage(e.target.value)}
                placeholder="Escribe un mensaje de prueba..."
                required
                style={{ padding: '8px', width: '100%' }}
              />
            </div>
            <button type="submit" style={{ width: 'fit-content' }}>Enviar Log</button>
          </form>

          {lastLogResponse && (
            <div style={{ marginTop: '16px' }}>
              <h4>Respuesta de envío de log:</h4>
              <pre>{JSON.stringify(lastLogResponse, null, 2)}</pre>
            </div>
          )}
        </section>
      </main>
    );
}

export default App;
```

Foto 5.4: Quinta captura del frontend

Muestra la interfaz del cliente (Frontend) junto con la consola de logs del sistema en ejecución. Se evidencia la interacción del usuario al generar una solicitud y la posterior confirmación recibida por la aplicación. En las trazas de los logs se valida el flujo del evento, confirmando la publicación correcta del mensaje dentro de la infraestructura de RabbitMQ.

Manejo de error en el Backend y derivación a DLQ (11 respuesta del backend sobre la orden fallida a dlq.jpg).

Explicación:

Muestra la respuesta emitida por el servidor (Backend) al procesar una orden que genera una excepción o fallo. Se aprecia la lógica de control de errores donde el sistema intercepta el fallo de procesamiento, rechaza el mensaje sin pérdida de información (nack/reject) y ejecuta la redirección automática del paquete hacia la Dead Letter Queue (DLQ) para su posterior inspección o reintento.

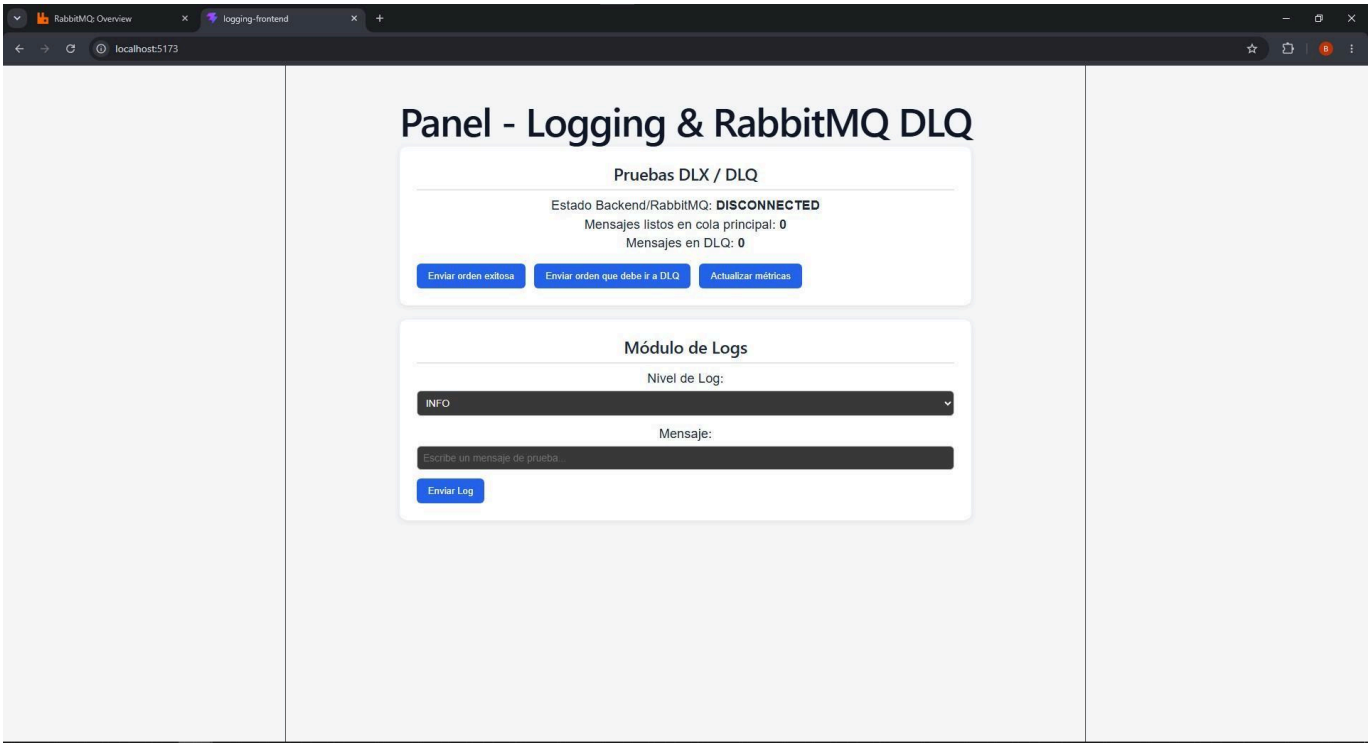


Foto 6: Captura del frontend desplegado

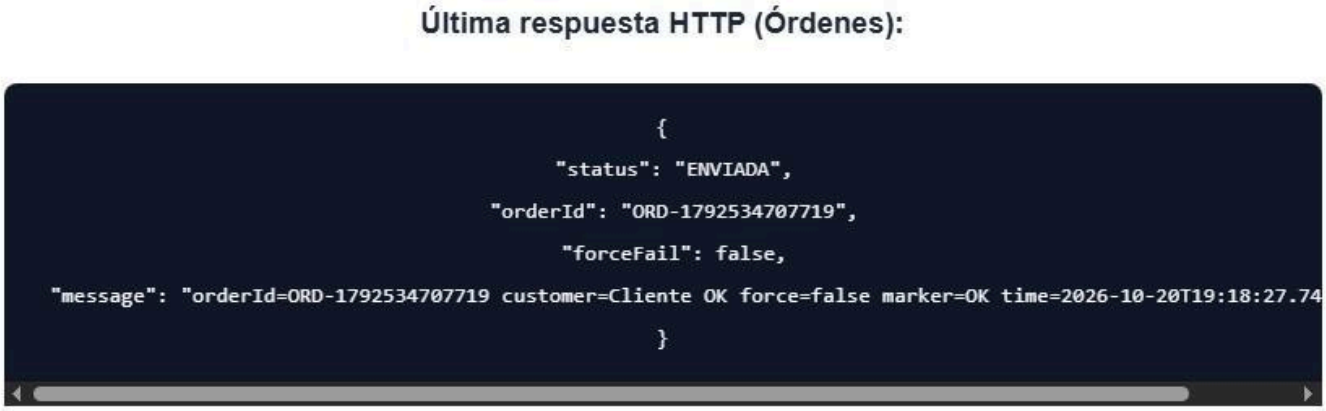


Foto 7: Confirmación del frontend al enviar la respuesta exitosa

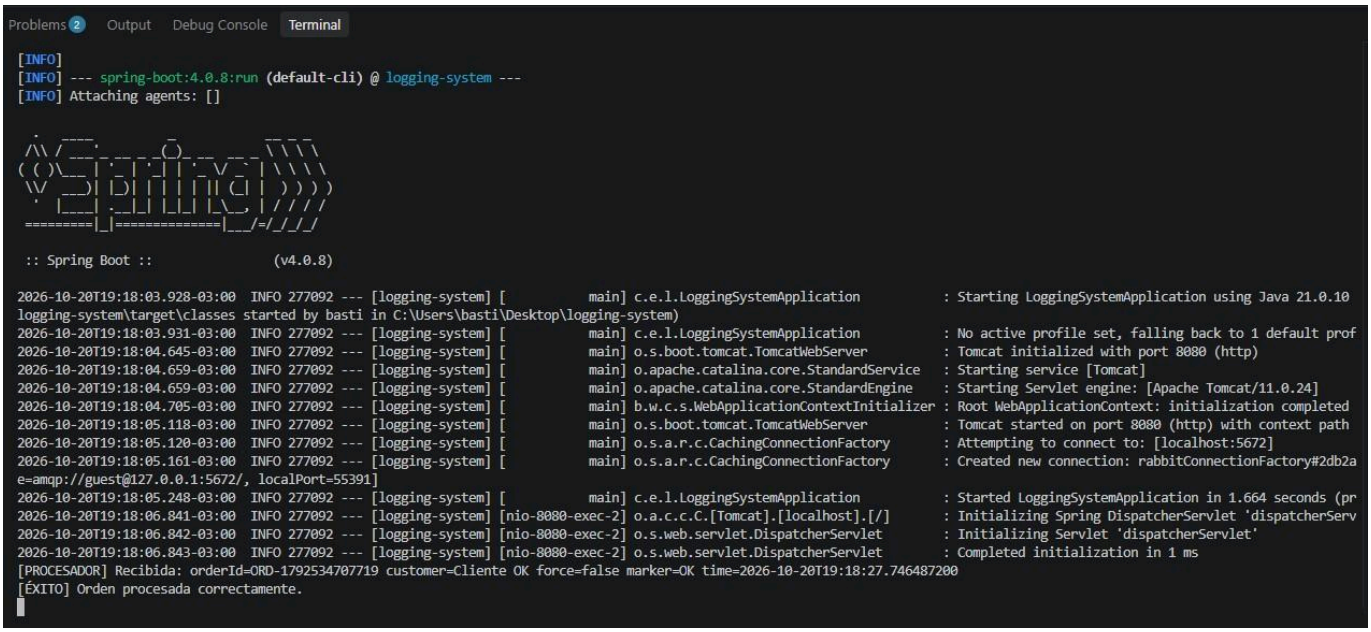


Foto 8: Respuesta del backend a la respuesta exitosa

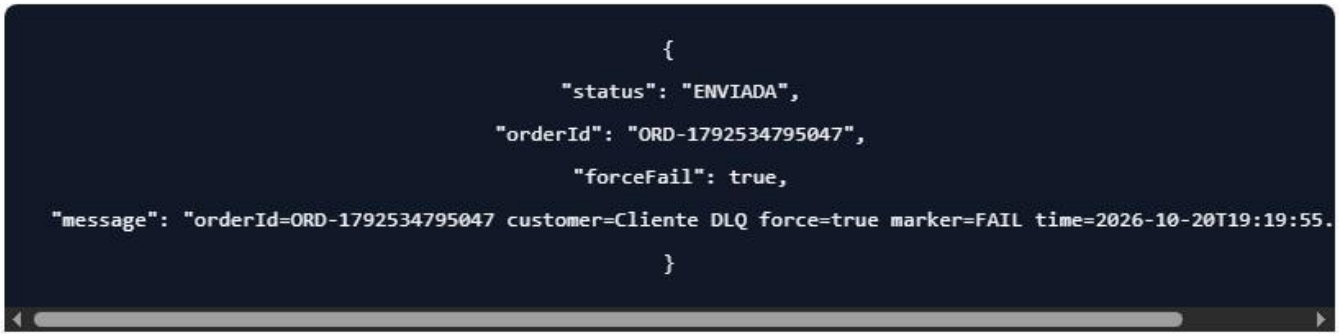


Foto 9: Respuesta del frontend al enviar una orden fallida a dlq



Foto 10: Respuesta del frontend sobre rabbitmq y logs

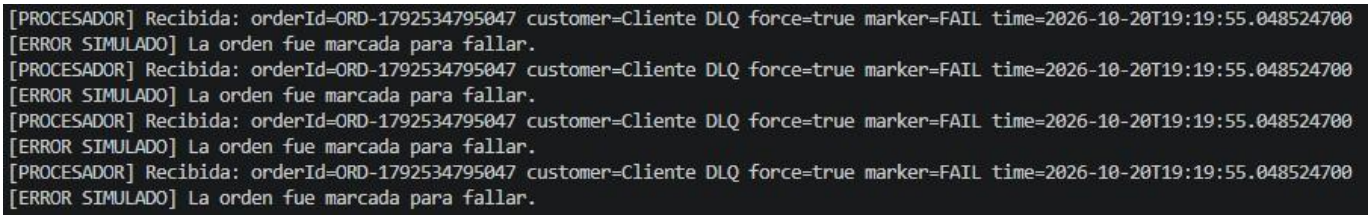


Foto 11: Respuesta del backend sobre la orden fallida a dlq

Estado de colas activas en la consola de RabbitMQ (12 evidencias de rabbitmq sobre las colas activas.jpg)

Explicación:

Muestra el panel de administración web de RabbitMQ (RabbitMQ Management Dashboard). En esta captura se aporta la evidencia de monitoreo del broker, mostrando la lista de colas activas, sus bindings, el volumen de mensajes (Ready, Unacked, Total) y el comportamiento gráfico del tráfico. Se verifica la coexistencia de la cola principal de procesamiento y la cola DLQ receptora de órdenes fallidas.

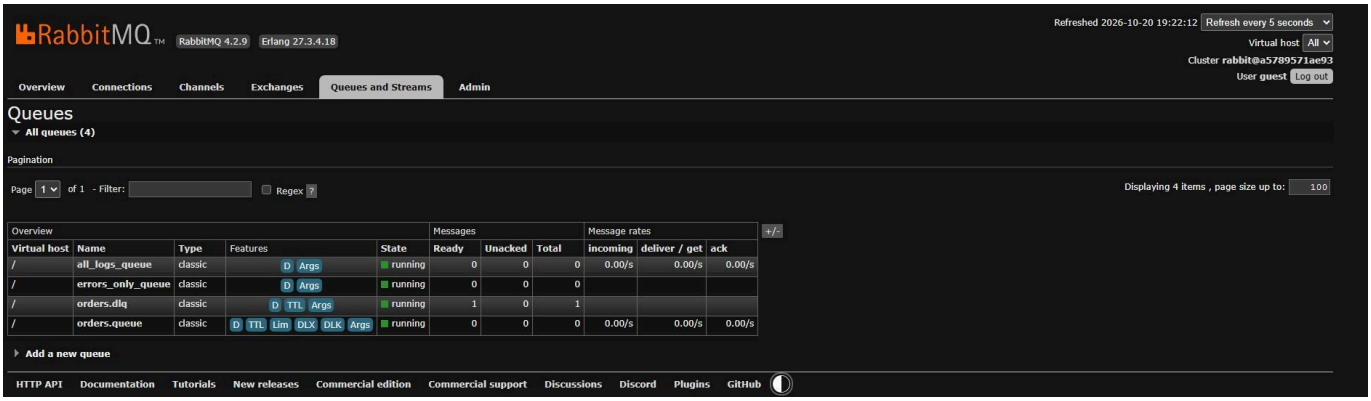


Foto 12: Evidencias de rabbitmq sobre las colas activas

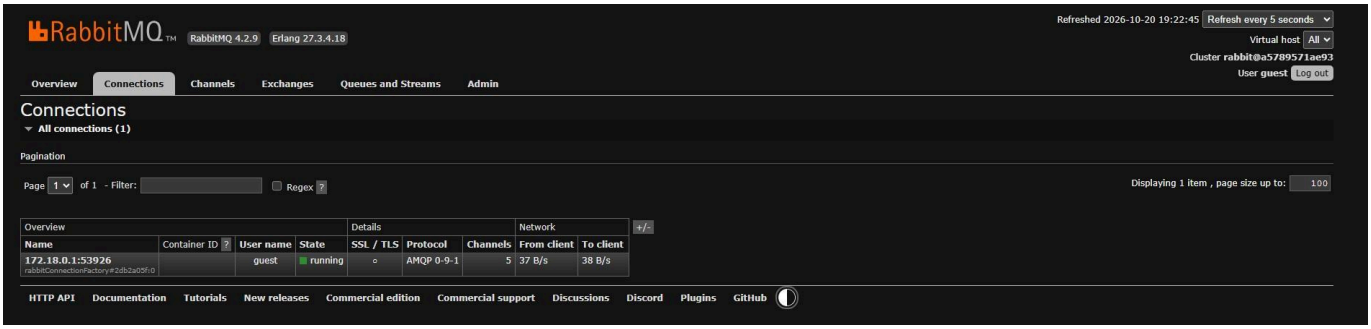


Foto 13: Evidencias de la conexión de rabbitmq

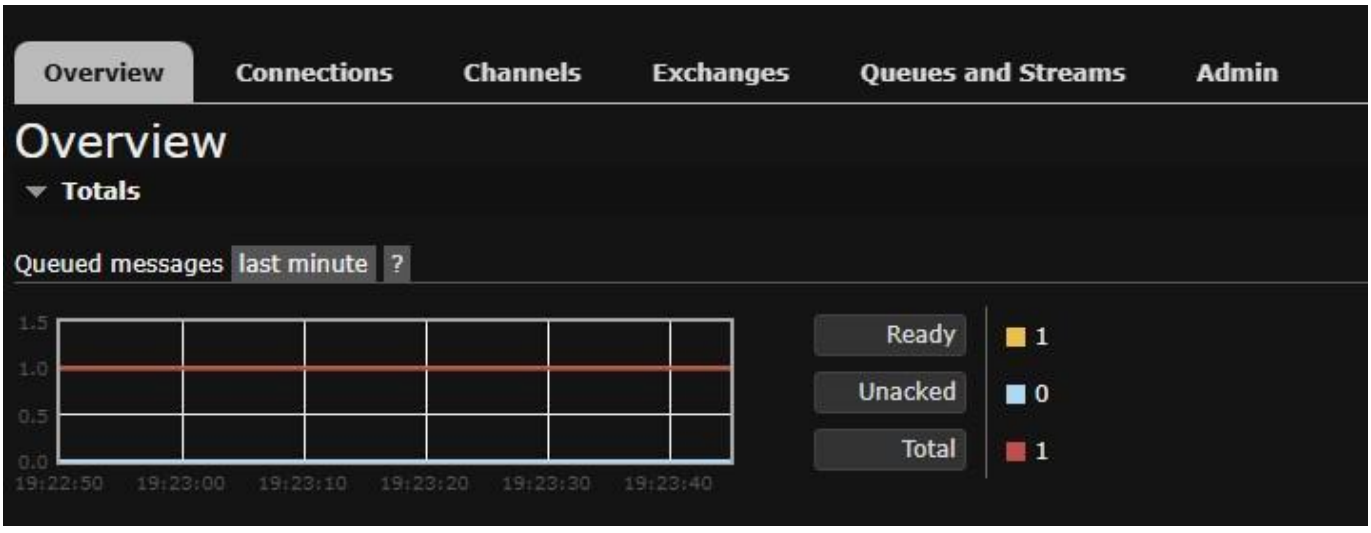


Foto 14: Evidencias de las colas de rabbitmq

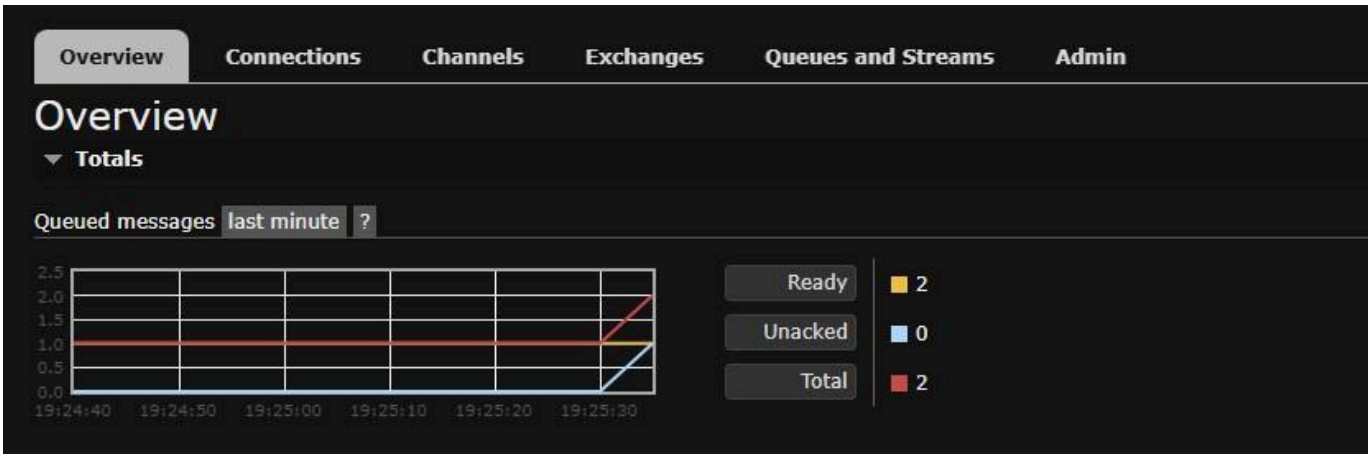


Foto 15: Más evidencias de las métricas de rabbitmq al enviar otra orden a dlq